Project Documentation

1. Introduction

- **Project Title:** Online Complaint Registration
- **Team Members:** Pulkit Khera

2. Project Overview

- **Purpose:** To provide a transparent, centralized web portal for users to easily submit, track, and manage grievances.
- **Features:** Secure user registration, automated complaint routing, real-time status tracking dashboard, and an administrative view for ticket management.

3. Architecture

- **Frontend:** The user interface is built using HTML5, CSS3, and JavaScript, utilizing Bootstrap for responsive design.
- **Backend:** The server-side architecture is powered by Python and the Flask framework to handle routing and application logic.
- **Database:** A relational SQL database (e.g., SQLite for development) is used, with SQLAlchemy ORM handling schema creation and database interactions.

4. Setup Instructions

- **Prerequisites:** Python 3.x, pip (Python package manager), and a local SQL database viewer.
- **Installation:**
    1. Clone the repository to your local machine.
    2. Create and activate a Python virtual environment.
    3. Run pip install -r requirements.txt to install dependencies.
    4. Run flask db upgrade to initialize the database schema.

5. Folder Structure

- **Client (Frontend):** The templates/ directory holds all HTML views (e.g., login.html, dashboard.html), while the static/ directory contains CSS stylesheets, JavaScript files, and images.
- **Server (Backend):** The core server logic is organized within the app/ directory, containing routes.py for endpoint management and models.py for database table definitions.

6. Running the Application

- Open your terminal in the project root directory.

- **Frontend/Backend:** Since Flask serves both the client and API, simply run flask run (or python app.py) in your terminal.
- The application will be accessible locally at http://127.0.0.1:5000.

7. API Documentation

- POST /register: Accepts email and password parameters to create a new user; returns a success redirect to the login page.
- POST /complaint: Accepts title, category, and description parameters from the form; generates and returns a unique Ticket ID.
- GET /dashboard: Fetches the authenticated user's tickets from the SQL database and renders the visual status view.

8. Authentication

- Authentication and authorization are handled using the Flask-Login extension for secure session management.
- User passwords are encrypted before database insertion using Werkzeug's security hashing functions.
- Private routes (like the dashboard) are secured using @login_required decorators.

9. User Interface

- *[Insert screenshot of the Login/Registration portal here]*
- *[Insert screenshot of the categorical Complaint Submission Form here]*
- *[Insert screenshot of the visual Status Tracking Dashboard here]*

10. Testing

- The testing strategy primarily utilizes manual User Acceptance Testing (UAT) following a predefined set of positive and negative test cases.
- The built-in Flask test client is utilized to mock requests and ensure backend routes return the correct HTTP status codes.

11. Screenshots or Demo

- A complete walkthrough video demonstrating the user journey from registration to ticket resolution can be viewed here: [Insert Link to Video/Demo].

12. Known Issues

- The system currently lacks frontend validation for file upload sizes, which may result in a crash if a user attempts to upload evidence larger than 5MB.

13. Future Enhancements

- Integration of automated email or SMS notifications to alert users when their ticket status changes.
- Implementation of a machine learning-driven chatbot to help users resolve common issues immediately without needing to open a ticket.